PROBLE v2

Secure Assessment Infrastructure

Full Team Execution Plan · 8-Person Role Assignment · Android + PC Secure Browser Shell

Pilot Institution: SRM University | Target: 8 Weeks

SECTION 1 — TEAM OVERVIEW

The 8-person team is structured into 3 tiers based on skill level. Tier 1 leads own core decisions and architecture. Tier 2 executors implement assigned tasks. Tier 3 support handles testing, documentation, and small polish items. The scope has been updated to include a custom PC Secure Browser (Electron shell) alongside the native Flutter Android application.

Person	Tier	Role	Primary Domain
Tharun	Lead	Tech Lead + Backend	NestJS · PostgreSQL · API Handshakes
Nithil	Lead	Realtime Systems Lead	Socket.IO · Redis · Real-time telemetry
Surjith	Lead	Mobile Lead	Flutter Android · Mobile Anti-Cheat
Sundar	Executor	Teacher Portal + PC Browser	React · Electron Shell · Kiosk Windowing
Aarya	Executor	Admin Portal + PC Security	React · Electron Security · Keyboard Hooks
Madan	Executor	DevOps + Infrastructure	Railway · CI/CD · Vercel Hosting
Bharath	Support	QA + Manual Testing	Test cases · Mobile & PC QA · VM Testing
Pugal	Support	UI Polish + Documentation	README · Seed data · Secure Installer Docs

SECTION 2 — LEAD ROLE CARDS

Tier 1 — Leads (Own decisions, architecture, code reviews)

THARUN — Tech Lead + Backend Lead

Owns: Overall architecture · NestJS backend · Database schema · API contracts · Cryptographic handshake validation

Why this role: Tharun is the project owner. The backend is the central dependency — every other system (Android, PC secure browser, portals) is blocked until API endpoints exist. The tech lead must own backend to unblock the entire team immediately.

Primary Responsibilities:

- NestJS project scaffolding and module architecture (12 modules)
- Database schema — TypeORM entities and Supabase PostgreSQL migrations
- Supabase JWT → NestJS Passport strategy (authentication layer)
- REST API endpoints: quizzes, attempts, import, gamification, analytics, admin
- Secure Handshake validation: verify custom header tokens from Android & PC Secure Browser
- Anti-cheat module: telemetry ingestion, threshold evaluation, session auto-flagging
- Code reviews for ALL pull requests (final merge authority)

- Unblocking Sundar and Aarya on API contracts and custom secure headers
- Swagger API documentation at /api/docs
- Technical tie-breaking decisions when the team disagrees

Does NOT own: Flutter code · Electron Native Desktop C++ hooks · Railway configuration

NITHIL — Realtime Systems Lead

Owns: Socket.IO gateway · Redis pub/sub · Timer synchronization · Reconnect logic · WebSocket reliability

Why this role: Realtime is the hardest engineering challenge in the entire system. Timer synchronisation failure = exam failure. This needs someone who can reason about distributed state, race conditions, and network failures. Cannot be delegated to a mid-level engineer.

Primary Responsibilities:

- Socket.IO gateway in NestJS (@nestjs/websockets)
- Upstash Redis adapter for Socket.IO (horizontal scaling ready)
- Room lifecycle: create → join → start → question release → end
- Server-driven timer — tick events broadcast from server every second (no client clock trust)
- Event acknowledgement system (guaranteed reliable delivery)
- Reconnect recovery — restore full exam state from PostgreSQL on reconnect
- Anti-cheat real-time events — teacher receives live flag:raised alerts from PC & Android clients
- Leaderboard real-time broadcasts via Redis sorted sets
- WebSocket stress testing and failure scenario validation (Week 7)
- Guides Sundar on which Socket.IO events to consume in teacher portal

Does NOT own: REST API endpoints · Database migrations · Mobile or PC Client UI code

SURJITH — Mobile Lead (Flutter Android)

Owns: Flutter Android application · Anti-cheat enforcement · Mobile device security · Exam session experience

Why this role: Flutter Android is highly specialized and fully isolated. Surjith can work independently from Week 1 using mock data without waiting for backend. Android anti-cheat requires native API knowledge — FLAG_SECURE, foreground services, overlay detection — that only a strong engineer can implement correctly.

Primary Responsibilities:

- Flutter project scaffold: GoRouter navigation, Riverpod state management
- Supabase Flutter SDK auth — login, register, secure JWT storage
- Dio HTTP client with JWT interceptor + cryptographic handshake header
- Socket.IO Flutter client connecting to Nithil's WebSocket gateway
- All mobile exam screens: code entry → lobby → secure exam session → results
- Anti-cheat service: FLAG_SECURE, immersive fullscreen, orientation lock

- AppLifecycle monitoring — background detection → telemetry event logged
- Telemetry buffer — batch events, flush to backend every 30 seconds
- Root detection (flutter_jailbreak_detection), emulator detection (device_info_plus)
- Foreground service — exam session survives app-switch attempts
- Clipboard clear on exam session start
- APK signing and internal distribution for SRM pilot devices
- Reviews Bharath's mobile test reports and fixes reported bugs

Does NOT own: PC Secure Browser shell (Electron) · Backend API code · Web portals

SECTION 3 — EXECUTOR ROLE CARDS

Tier 2 — Executors (Implement assigned tasks, ask leads when uncertain)

SUNDAR — Teacher Portal & PC Browser Shell Developer

Owns: proble-teacher React app · Custom PC Secure Browser shell (Electron windowing)

Reports to: Tharun for API / Desktop questions · Nithil for WebSocket questions

Why this role: Sundar is strong in React/JS. Building the PC secure browser wrapper in Electron uses the same web stack, allowing him to easily scaffold the browser container while managing the teacher portal.

Primary Responsibilities:

- Set up proble-teacher as a standalone Vite + React + TypeScript project
- Scaffold the PC Secure Browser project using Electron JS
- Electron Windowing: enforce Fullscreen Kiosk Mode (no window resizing, always-on-top, hide taskbars)
- Auto-launch configurations: block window exit shortcuts (Alt+F4, swipe-away controls)
- Axios HTTP client configured to point at NestJS backend from both Teacher portal and Electron shell
- Port existing pages off direct Supabase calls: Dashboard, QuizCreate, LiveTests, Master, Global
- Implement LiveController using Socket.IO (Nithil provides event specs)
- LiveLobby page with real-time student presence grid
- Live monitoring panel: flagged alerts feed + answer distribution charts
- Import wizard: Excel upload + ZIP media upload with progress tracking

AARYA — Admin Portal & PC Browser Security Developer

Owns: proble-admin React app · Custom PC Secure Browser security modules · Shared UI component library

Reports to: Tharun for API questions

Why this role: Aarya manages admin tools and shared interfaces. In the PC Secure Browser, she implements the OS-level hook security scripts (keyboard interceptors, process scans) that keep the PC client locked down.

Primary Responsibilities:

- Set up proble-admin as a standalone Vite + React + TypeScript project
- PC Secure Browser Keyboard Hooking: intercept system-level keys (Alt+Tab, Win Key, Cmd+Tab, PrintScreen)
- PC Secure Browser Process Guard: scan active Windows/macOS processes for screen recorders and remote desktop software (AnyDesk, Zoom, TeamViewer)
- PC Secure Browser Anti-VM Shield: detect virtual machine environments (VirtualBox, VMware) on desktop launch
- Inject secure client headers/tokens to validate the PC client identity to NestJS
- Shared UI component library: Button, Input, Table, Card, Modal, Chart wrappers
- Admin pages: User management, Institution config, Maintenance mode toggle
- Audit log viewer: searchable, filterable immutable event trail table
- Device trust management page: list of registered devices (Mobile + PC) with trust scores
- Recharts analytics components used by both teacher and admin portals

MADAN — DevOps + Infrastructure

Owns: Railway deployment · GitHub Actions CI/CD · Environment management · Monitoring

Reports to: Tharun for configuration questions

Critical rule: Madan does NOT modify backend TypeScript or Electron code. He only touches configuration files, CI/CD scripts, and infrastructure settings.

Primary Responsibilities:

- Railway account + NestJS backend service: configured and deployed from GitHub
- Upstash Redis instance provisioned and environment variable connected
- GitHub Actions pipeline: lint → test → build → deploy on main branch push
- Environment variables management: Railway secrets, Vercel env vars
- Vercel projects for teacher and admin portals (auto-deploy from GitHub)
- Staging environment: separate Railway service for safe pre-production testing
- Monitoring alerts: Railway metrics, Supabase dashboard, Upstash metrics
- Custom domain setup (api.proble.in), SSL certificate verification
- Weekly: run npm audit, check dependency vulnerabilities, report to Tharun
- Deployment runbook documentation for the team
- k6 load test execution (Tharun writes scripts, Madan runs them)

SECTION 4 — CUSTOM PC LOCKDOWN BROWSER (PROBLE SECURE BROWSER)

To raise cheating difficulty to an institutional trust level on laptops and desktops, the team is building a custom PC Lockdown Browser (Electron JS shell) instead of relying on standard Chrome/Safari or third-party config files. The student React Web App loads directly inside this secure wrapper.

Core Lockdown Mechanism:

- Kiosk Mode Enforcement: Launches in fullscreen, always-on-top, and disables standard OS borders/close buttons. Students cannot minimize or swipe the application away.
- System Shortcut Blocking: Electron intercept APIs intercept global keyboard hooks. Alt+Tab, Windows Key, Cmd+Tab, Ctrl+Alt+Del, and PrintScreen are completely blocked during active quiz sessions.
- Clipboard Cleanser: Wipes the OS clipboard continuously during the session to block copy-paste leakage.
- Process Scan & Block: Node.js native child process scans. If screen sharing (AnyDesk, TeamViewer, Zoom), virtual machines (VMware, VirtualBox), or developer tools are active, the exam locks and registers a violation event.
- Cryptographic Backend Gatekeeper: Inject a signed client verification token inside request headers. NestJS uses a custom Guard to reject any student attempt to access quiz endpoints using a standard browser (like standard Google Chrome).

SECTION 5 — SUPPORT ROLE CARDS

Tier 3 — Support (Concrete non-architecture tasks, always ask when scope is unclear)

BHARATH — QA + Manual Testing

Owns: Test cases · Bug reports · Physical device testing · PC browser lock-down verification

Reports to: Feature owners (Surjith for Android, Sundar/Aarya for PC shell)

Concrete tasks Bharath executes without needing to code:

- Maintain Google Sheet of 50+ test cases covering Mobile and PC environments
- Install APK on Android device and test E2E quiz lifecycle
- Download and install PC Secure Browser (.exe/.dmg) -> verify kiosk mode holds
- Attempt to cheat on PC: press Alt+Tab, printscreen, run AnyDesk, run virtual machines -> verify blocks hold and telemetry is sent
- Open standard Chrome and attempt to access quiz -> verify NestJS blocker is active
- Disconnect WiFi mid-exam on both PC and Android -> verify local answers auto-save and restore upon reconnect
- Cross-browser testing: teacher portal on Chrome, Firefox, Edge
- File GitHub issues with: steps to reproduce + screenshot + client logs

PUGAL — UI Polish + Documentation + Data Seeding

Owns: Visual polish tasks · README and docs · Sample data · Secure Installer Guides

UI tasks (assigned by Surjith or Sundar):

- Ensure matching color scheme, margins, and loading indicators between Android app and PC Secure Browser UI
- Implement skeleton loaders on list, portal, and dashboard screens
- Ensure proper error dialog displays inside the PC shell when the network goes down

Documentation & Seeding tasks:

- Write and maintain README.md for all project repositories (including the new PC client)
- Create the student PC installer guide: clear instructions on downloading and launching "Proble Secure Browser"
- Teacher onboarding guide: walkthrough showing how to monitor PC & Mobile telemetry violations
- Create SRM seed data: 1 organisation record, 5 faculty accounts, 30 student accounts
- Import first SRM question bank from Excel to verify import pipeline

SECTION 6 — 8-WEEK SPRINT BREAKDOWN

The sprint table has been updated to include the parallel design and deployment of the custom PC Secure Browser.

Weeks 1–2: Foundation

Person	Tasks — Weeks 1–2
Tharun	NestJS scaffold · Supabase JWT strategy · TypeORM entities · Users + Quizzes CRUD · Run migrations 001–005 · Define custom PC Client header specs
Nithil	Socket.IO gateway setup · Upstash Redis connection · Room create/join primitives · Basic heartbeat + presence gateway
Surjith	Flutter project scaffold · GoRouter + Riverpod · Supabase auth screens · Dio client setup · Basic navigation flow
Sundar	proble-teacher Vite setup · Scaffold PC Secure Browser (Electron project setup) · Electron window launcher in basic kiosk window
Aarya	proble-admin Vite setup · Shared UI component library · Admin layout skeleton · Start Electron system key hook research
Madan	Railway NestJS deployment · Upstash provision · GitHub Actions CI/CD · Vercel teacher + admin projects
Bharath	Write 50 test case scenarios in Google Sheet (Mobile + PC) · Set up test environment · Explore existing app behaviour
Pugal	README for all repos · Supabase seed: 1 org (SRM) + 5 faculty + 30 students · Prepare demo quiz content

Weeks 3–4: Core Exam Flow

Person	Tasks — Weeks 3–4
Tharun	Attempts module (start/autosave/submit/score) · Import module (SheetJS Excel parsing) · Supabase Storage upload · Question randomisation
Nithil	Full room lifecycle (start→question release→end) · Server-driven timer · WS answer submission · Reconnect recovery · Event ACKs
Surjith	Exam lobby screen · Exam session (questions, MCQ, server timer display) · Submit + results · FLAG_SECURE + immersive mode
Sundar	LiveController migrated to Socket.IO · LiveLobby with presence grid · Embed student web app inside the Electron kiosk window
Aarya	Admin user list page · Recharts chart components · Import wizard UI · Complete Electron OS system hook shortcut blocking (Alt+Tab, Win)
Madan	Staging environment on Railway · k6 load test setup (50 concurrent WS connections) · Monitoring alerts configured
Bharath	Test full exam flow end-to-end (Mobile + basic PC shell) · Test reconnect scenario · Test import · File bugs on GitHub
Pugal	Draft teacher onboarding guide · Create 50-question SRM sample Excel · Screenshot and report any UI issues

Weeks 5–6: Anti-Cheat + Security

Person	Tasks — Weeks 5–6
Tharun	Anti-cheat module (telemetry ingestion + threshold evaluator + auto-flag) · PC Client handshake verification Guard · Gamification · Audit log
Nithil	Leaderboard real-time broadcasts · Teacher flag:raised live events (Mobile & PC events) · Force-end session · Anti-cheat live feed via WS
Surjith	Telemetry buffer (30s batch flush) · AppLifecycle monitoring · Root detection · Emulator detection · Foreground service
Sundar	Live monitor panel (flag feed + student grid) · Answer distribution charts · Full analytics page · Electron kiosk exit protection
Aarya	Admin audit log viewer · Device trust management page · Anti-cheat policy config · Electron Process Scanner & VM Shield scripts
Madan	Rate limiting (NestJS throttler) · CORS whitelist · npm audit clean · Security headers configured on Railway
Bharath	Anti-cheat manual tests: switch apps, try screenshot, try PC Alt+Tab, run VM, run AnyDesk → verify blocks & alerts hold
Pugal	Student PC secure browser installer guide · Demo video script · Fix UI polish items from Bharath bug reports

Week 7: Integration + Stress Testing

Person	Tasks — Week 7
Tharun	API hardening · Input validation audit · Swagger docs complete · Fix backend bugs from integration testing
Nithil	8-hour WS stability test · Reconnect stress testing · Timer drift investigation and fix · WS error edge cases
Surjith	Physical device testing on 10+ Android phones · APK signing · Fix Flutter bugs from Bharath reports
Sundar	Cross-browser testing (Chrome/Firefox/Edge) · Fix teacher portal UI bugs · UX polish pass · Compile final desktop installer shell
Aarya	Admin portal UAT · Fix analytics data display bugs · Responsive layout fixes on small screens · Test PC Client OS hooks on Windows 10/11
Madan	50-student load test on staging (Mobile + PC simulations) · Production deploy rehearsal · Verify backup strategy works
Bharath	Full regression: all 50 test cases (Mobile & PC Secure Browser) · Record pass/fail status · Priority bug list for Tharun
Pugal	Loom walkthrough recording for SRM faculty · Final README review · Confirm all demo quizzes work on staging

Week 8: SRM Pilot Launch

Person	Tasks — Week 8
Tharun	Production deploy sign-off · Critical bug fixes only · SRM faculty account creation and setup
Nithil	Production WebSocket monitoring · On-call during first SRM live exam session
Surjith	APK distribution to 30 SRM students · Install support · On-call during exam for mobile issues
Sundar	Teacher portal walkthrough with SRM faculty · Fix any UX issues discovered during demo
Aarya	Admin portal live for SRM administrator · System health monitoring during pilot
Madan	Monitor Railway + Supabase metrics during live exam · Respond to any infrastructure alerts
Bharath	Live QA observer during SRM exam: note all issues in real time, report immediately
Pugal	Support students with PC installer/APK install questions · Document post-pilot issues for v1.1 backlog

SECTION 7 — OWNERSHIP BOUNDARIES

Clear ownership eliminates coordination overhead. These boundaries are enforced strictly.

Person	Owns (can freely modify)	Cannot modify without permission
Tharun	proble-backend/src/** (except realtime/) · All DB migrations · API contracts & PC/Mobile signature keys	Flutter code · Electron wrapper · Railway config · WebSocket gateway internals
Nithil	proble-backend/src/realtime/** · Redis configuration	REST API endpoints · Database migrations · Mobile or PC Client UI code
Surjith	proble-android/lib/** · proble-android/android/**	Backend code · PC Secure Browser (Electron) · Web portals
Sundar	proble-teacher/src/** · proble-secure-browser/src/main/** (Electron window skeleton)	Backend code · Admin portal · PC security scripts (Aarya's domain)
Aarya	proble-admin/src/** · shared-ui/src/** · proble-secure-browser/src/security/** (PC hooks)	Backend code · Teacher portal · PC windowing skeleton (Sundar's domain)
Madan	.github/workflows/** · railway.json · docker-compose.yml · nginx.conf	Any TypeScript/Dart source code
Bharath	testing/test-cases.xlsx · GitHub bug report issues	Any source code (testing role only)
Pugal	*/README.md · docs/** · seed-data/**	Any production source code

SECTION 8 — COORDINATION RULES

Daily Standup — 15 Minutes Every Morning

Each person answers three questions: (1) What did I finish yesterday? (2) What am I working on today? (3) Am I blocked by anyone?

GitHub Workflow

- Every feature → separate branch from main (no direct push to main)
- Pull request required for ALL merges
- Tharun reviews Backend, Surjith reviews Flutter Android, Nithil reviews WebSockets, Sundar/Aarya review PC Secure Browser/Portals
- Bharath tests every PR on staging before it merges to main

Feature Freeze Rule

After Week 4: NO new features are added. Weeks 5–8 are strictly for completing, security testing, and stress-testing what was planned. This rule is enforced by Tharun and cannot be overridden.

SECTION 9 — RISK REGISTER

Risk	Owner	Mitigation
Students bypass PC lockdown	Tharun	NestJS checks secure client

by opening standard web browser		signature key. Requests from regular browsers like Chrome are blocked.
WebSocket timer drift causes exam inconsistency	Nithil	Server-only timer. No client clock trust. 8-hour stability test in Week 7.
Android APK or PC Secure Browser blocked by SRM IT	Surjith / Sundar	Present security keys to SRM IT in Week 6. Provide direct download mirrors on SRM local servers.
Railway free tier cold starts delay WebSocket connections	Madan	Upgrade to Railway $5/month Hobby plan before pilot. Implement keep-alive ping every 25s.
Students use Virtual Machines to bypass PC Secure Browser hooks	Aarya	Add BIOS and motherboard string checks inside the Electron launcher to completely block VM execution.
Feature creep from team members suggesting additions	Tharun	Hard feature freeze after Week 4. All suggestions go to a V2+ backlog.

SECTION 10 — V2+ ROADMAP (POST-SRM PILOT)

These features are explicitly OUT OF SCOPE for the 8-week sprint.

Feature	Priority	Estimated Effort
Play Integrity API (device attestation)	High	1 week
Play Store and macOS App Store public release	High	2 weeks
AI question generation via Gemini API	High	2 weeks
Placement preparation modules	High	3 weeks
AWS migration (EC2 + RDS + ElastiCache)	Medium	1 week
iOS native security app	Medium	2 weeks
Front-camera proctoring snapshots	Low	3 weeks